

Analizador léxico y sintáctico de C-TDS

Diseño e interfaz del AST

Agustin Alieni, Fran Natale y Julian Varea

1. Introducción

En esta etapa construimos el analizador léxico (`lexer.l`, con Flex) y el analizador sintáctico (`bison.y`, con Bison) de C-TDS, y dejamos definida la interfaz del árbol de sintaxis abstracta (AST) que va a sostener las etapas siguientes del compilador (análisis semántico, generación de código intermedio y, finalmente, assembly). En esta entrega no implementamos el AST: dejamos declarada su interfaz en `ast.h`, con los cuerpos de `ast.c` vacíos, y las acciones semánticas de la gramática comentadas, a la espera de que terminemos de cerrar en equipo algunas decisiones de diseño que todavía están abiertas.

2. División del trabajo

El trabajo de esta etapa se dividió en dos frentes, que avanzaron en paralelo y se integraron sobre el final:

- **Analizador léxico:** definición y ajuste de las expresiones regulares de `lexer.l` para cubrir todos los tokens de la especificación, y de los mensajes de error léxico.
- **Analizador sintáctico y AST:** diseño de la gramática de `bison.y`, detección y resolución del conflicto shift/reduce descrito en la Sección 4, y diseño de la interfaz del AST en `ast.h`.

Las decisiones de diseño más importantes (la resolución del conflicto shift/reduce y el criterio para el AST) se discutieron y acordaron en conjunto entre los tres integrantes antes de llevarlas al código, para no depender de que una sola persona las resolviera por su cuenta.

3. Analizador léxico

El lexer reconoce todos los tokens que describe la especificación del lenguaje: las palabras reservadas (`boolean`, `int`, `float`, `void`, `if`, `else`, `while`, `return`), los identificadores, las constantes numéricas (enteras y de punto flotante), las constantes booleanas, los operadores aritméticos, relacionales y lógicos, los delimitadores, y los comentarios de línea y multilinea.

3.1. `main` no es palabra reservada

Decidimos no tratar `main` como palabra reservada, a pesar de que el lenguaje exige que todo programa tenga un método con ese nombre. La razón es que esa exigencia es una regla semántica (debe existir un método llamado `main` con una firma particular), no una regla léxica ni sintáctica: en la gramática, `main` es simplemente el identificador de un método como cualquier otro. Si lo hubiéramos declarado como palabra reservada, un identificador de esa forma dejaría de poder usarse como nombre de variable o de cualquier otro método, lo cual no está pedido en ningún lado de la especificación. Por eso `main` matchea la regla general de `ID`, y la verificación de que exista un método `main` con la firma correcta queda para el análisis semántico.

3.2. Número de línea en los mensajes de error

Activamos `%option yylineno` y lo usamos tanto en los mensajes de error léxico del propio `lexer.l` como en `yyerror` del lado de Bison. Nos pareció imprescindible para que un mensaje de error sea útil: sin número de línea, encontrar dónde está el problema en un archivo fuente de varias decenas de líneas obliga a revisar todo el archivo a mano.

4. Decisiones de diseño, asunciones y extensiones

4.1. Un conflicto shift/reduce que nos obligó a rediseñar la gramática

La primera versión de la gramática que escribimos separaba las declaraciones globales en dos listas independientes, una para variables y otra para métodos:

```
Program : VariableDeclarations MethodDeclarations ;
```

Esta estructura calca directamente el orden en que aparecen en la letra los no terminales `<var_decl>` y `<method_decl>`, pero no funciona en la práctica: al probarla contra el programa de ejemplo de la propia especificación del lenguaje, que declara una variable global y a continuación un método con retorno `no-void`,

```
int a;
int sumar(int a, int b) { }
```

el parser fallaba con un error de sintaxis apenas encontraba el segundo `int`. La causa es un conflicto de desplazamiento/reducción (shift/reduce) inherente a un parser LALR(1), que solo tiene un token de adelanto (lookahead) para decidir. Tanto una declaración de variable como una de método empiezan igual, con `Type ID`, y Bison, ante la duda, siempre prefiere desplazar antes que reducir: al ver el segundo `int` todavía dentro de `VariableDeclarations`, asumía que se trataba de otra variable y esperaba un `;` o una `,`. Como en cambio encontraba un `(`, la lista de variables nunca llegaba a cerrarse y el parser no tenía forma de retroceder para reinterpretar esa declaración como un método. En la práctica, esto significa que ningún método con tipo de retorno `no-void` lograba parsearse si estaba precedido por al menos una variable global, lo cual invalidaba directamente el ejemplo de la letra.

La solución que aplicamos fue unificar ambas listas en una sola, dejando que la decisión de si `Type ID` arranca una variable o un método se tome recién al ver el token siguiente:

```
Declarations
: /* vacio */
| Declaration Declarations
;
```

```
Declaration
: VariableDeclaration
| MethodDeclaration
;
```

Con esta forma, el parser consume `Type ID` de manera neutra, sin comprometerse todavía a ninguna de las dos alternativas, y recién decide al ver el token siguiente: si es `;` o `,`, se trata de una variable; si es `(`, de un método. Esto permite además que variables y métodos aparezcan intercalados en el scope global, que es exactamente lo que hace el ejemplo de la letra. De paso, aprovechamos para sacar el no terminal intermedio `ReturnType` e integrar directamente `Type` y `VOID` en `MethodDeclaration`, para no introducir una reducción adicional que pudiera volver a confundir al parser en ese mismo punto.

4.2. Precedencia de operadores

Para resolver la precedencia y asociatividad de los operadores de **Expression** usamos las declaraciones `%left` y `%right` de Bison, siguiendo la tabla de precedencia de menor a mayor que se desprende de la especificación del lenguaje: disyunción lógica, conjunción lógica, igualdad, relacionales, suma y resta, multiplicación/división/resto, y por último la negación lógica y el menos unario. Optamos por esta vía declarativa en lugar de reescribir **Expression** como una jerarquía explícita de no terminales por nivel de precedencia (una alternativa habitual cuando no se cuenta con declaraciones de precedencia de la herramienta).

El menos unario y `%prec UMINUS`. El símbolo `-` cumple dos roles con precedencias distintas: la resta binaria (nivel de `+` y `-`) y el menos unario (nivel más alto, junto con `!`). El lexer no puede distinguirlos, porque ambos son el mismo carácter, así que siempre devuelve el mismo token `'-'`. Bison, por su parte, le asigna a cada regla la precedencia de su último terminal, por lo que la regla `'-' Expression` heredaba el nivel de la resta binaria. En consecuencia, al ver `-a * b` el parser prefería desplazar el `*` y agrupaba `-(a * b)`, en lugar de `(-a) * b` como indica la tabla de la especificación (lo mismo con `/` y `%`).

Para corregirlo declaramos un token ficticio **UMINUS** en el nivel más alto, que el lexer nunca devuelve y que existe solo para ocupar ese nivel de precedencia, y se lo asignamos a la regla con `%prec`:

```
%right NOT UMINUS
...
| '-' Expression %prec UMINUS
```

Es el mecanismo estándar de Bison para la precedencia dependiente del contexto. Es la única regla de la gramática que lo necesita, porque `-` es el único token con dos roles: la regla **NOT Expression** no lo requiere, ya que su último terminal es **NOT**, que ya está en el nivel más alto. El cambio no modifica el conjunto de programas aceptados: solo determina cómo se agrupan las expresiones y, por lo tanto, la forma del árbol que construirán las acciones semánticas. Por eso conviene resolverlo en el parser y no posponerlo al AST, que hereda la agrupación en el orden en que Bison reduce.

4.3. Condición del while entre paréntesis

En la gramática de la especificación, la sentencia **while** aparece como **while** `<expr>` `<block>`, sin paréntesis alrededor de la condición, a diferencia del **if**. De acuerdo con lo hablado en clase, se trata de un error de la especificación, así que tomamos la condición del **while** entre paréntesis, igual que en el **if**:

```
| WHILE '(' Expression ')' Block
```

4.4. Comentarios multilínea sin cerrar

Asumimos, por el momento, que todo comentario multilínea que se abre en el código fuente se cierra correctamente. El caso de un comentario multilínea que nunca se cierra (el archivo termina dentro de un comentario abierto) queda señalado como algo pendiente de resolver en una iteración posterior del lexer, y se detalla en la Sección 8.

4.5. AST: solo interfaz en esta etapa

Decidimos, como extensión al alcance mínimo pedido para esta entrega, dejar completamente definida y documentada la interfaz del AST (`ast.h`), aunque no forma parte de lo exigido para el analizador sintáctico en sí. La justificación de esta decisión se desarrolla en la Sección 5.

5. Descripción de diseño y justificación

5.1. Alternativas consideradas para el conflicto shift/reduce

Antes de llegar a la unificación de `Declarations/Declaration` descrita en la Sección 4, consideramos dos alternativas que descartamos:

- **Aumentar el lookahead:** Bison genera parsers LALR(1), con un solo token de adelanto; no hay forma de pedirle más lookahead sin cambiar de herramienta o de clase de gramática, así que esta alternativa no era viable con `lex/yacc`.
- **Forzar el orden en la letra** (todas las variables antes que los métodos): en teoría resolvía el conflicto, pero invalidaba el propio ejemplo de programa que trae la especificación del lenguaje, que intercala una variable y un método. La descartamos porque no es fiel al lenguaje que tenemos que reconocer.

La alternativa que aplicamos (unificar ambas listas en un único no terminal y diferir la decisión hasta ver el token siguiente a `Type ID`) es el patrón estándar para este tipo de ambigüedad de prefijo compartido en gramáticas LL/LALR, y es la que finalmente seleccionamos porque no exige tocar el lenguaje ni la herramienta, solo la forma en que factorizamos la gramática. La verificamos de dos formas: generando la gramática con `bison -d -v` y confirmando que el archivo de reporte no muestra ningún conflicto shift/reduce ni reglas inalcanzables (algo que sí aparecía con el diseño original), y compilando el parser completo contra Flex y la interfaz de AST para correrlo sobre el ejemplo de la letra y sobre variantes propias (variables intercaladas con métodos, condicionales anidados, ciclos, llamadas a método con y sin argumentos), confirmando que todos los casos válidos parsean sin errores formales.

5.2. Por qué separamos el AST de `bison.y`

Decidimos no definir la estructura del AST directamente en `bison.y`, sino en un archivo aparte, `ast.h`. La razón es que sobre este árbol todavía se va a construir el análisis semántico, un intérprete o generador de código intermedio, y finalmente el generador de assembly, y no queríamos que esas etapas dependieran de un archivo pensado en primer lugar para la gramática. Separar la interfaz del árbol en su propio header nos permite mantener `bison.y` enfocado en las reglas de producción y dejar la definición de los nodos como un contrato aparte, que cualquier etapa posterior puede incluir sin arrastrar nada de Bison.

5.3. Por qué `ast.h` solo tiene interfaz, y `ast.c` está vacío

Dentro de esa separación, distinguimos además interfaz de implementación: `ast.h` declara los tipos y las firmas de las funciones constructoras del árbol, documentando en comentarios qué hace cada una, quién es dueño de la memoria que devuelve y qué se espera de sus parámetros; `ast.c` contiene esas mismas funciones pero con el cuerpo vacío (un `TODO`). No las implementamos todavía porque hacerlo implica tomar decisiones de diseño que como equipo no terminamos de cerrar (por ejemplo, si el árbol va a liberar memoria a medida que se descarta, o cómo van a convivir exactamente los punteros `left/mid/right` con el arreglo `children[]` en los distintos tipos de nodo). Preferimos dejar esas firmas fijas y bien documentadas ahora, y resolver el cuerpo de cada función más adelante, cuando todo el equipo esté de acuerdo en cómo hacerlo, en lugar de que esa decisión quede tomada de forma implícita por quien haya escrito el primer código. Por eso en esta entrega `ast.h` es, en los hechos, toda la documentación de diseño del AST que tenemos: cada función está comentada con su propósito, sus parámetros y quién es responsable de liberar la memoria que devuelve, aunque todavía no haga nada.

Por el mismo motivo dejamos las acciones semánticas de `bison.y` escritas pero comentadas: cada regla de la gramática indica, a modo de comentario, cómo se vería la construcción del nodo

correspondiente una vez que el AST esté implementado (qué **NodeType** le corresponde, qué va en **left/mid/right** o en **children[]**), pero no se ejecuta ningún código todavía. Esto nos permitió dejar registrada, regla por regla, la intención de diseño de cada construcción del lenguaje sin depender de que **ast.c** ya esté terminado, y sin arriesgarnos a que el parser genere nodos a medio definir o haga referencias a memoria sin inicializar. Cuando decidamos avanzar con la implementación del AST, el trabajo va a consistir en completar los cuerpos de **ast.c** siguiendo lo que ya documentamos en **ast.h**, y recién ahí descomentar estas acciones.

5.4. Estructura de NodeAST

Cada nodo del árbol (**NodeAST**) tiene un **NodeType** que indica qué construcción del lenguaje representa, un **Symbol** opcional para guardar el identificador o el valor asociado, punteros **left**, **mid** y **right** para los casos con una cantidad fija y chica de hijos (por ejemplo, una asignación o un **if-else**), y un arreglo **children[]** de tamaño variable para los casos donde la cantidad de hijos depende del programa fuente (por ejemplo, la lista de identificadores de una declaración de variables, o los argumentos de una llamada a método). El campo **type** guarda el tipo semántico del nodo (**TYPE_INT**, **TYPE_BOOL**, etc.), separado de **nodeType** para no mezclar “qué construcción sintáctica es este nodo” con “qué tipo de dato tiene”, que son dos preguntas distintas y que además se resuelven en momentos distintos del proceso (la primera durante el parsing, la segunda recién en el análisis semántico).

5.5. Criterio para crear un nodo

No decidimos crear un nodo por cada no terminal de la gramática, sino únicamente para las construcciones que aportan información propia o que cambian el comportamiento del árbol resultante. Hay no terminales que existen en la gramática solo para organizar la sintaxis (evitar ambigüedades, factorizar alternativas) pero que en el AST no necesitan representación propia: por ejemplo, **Statement** agrupa todas las variantes de sentencia posibles, pero el nodo real lo termina aportando la alternativa concreta que *matcheó* (una asignación, un **return**, un bloque), así que **Statement** simplemente redirige el valor de su hijo hacia \$\$ sin crear un nodo nuevo. En cambio, **Block** sí tiene su propio **BLOCK_NODE**, porque no es un simple contenedor sintáctico: representa la apertura de un nuevo scope y agrupa las declaraciones de variables y las sentencias que contiene, que sí son información real que el árbol necesita conservar.

6. Detalles de implementación interesantes

6.1. Resolución de la ambigüedad Type ID

El punto más particular de esta etapa es, en términos de implementación, la técnica usada para resolver la ambigüedad de prefijo compartido entre **VariableDeclaration** y **MethodDeclaration** (ver Sección 4). No es un algoritmo en el sentido clásico, sino una técnica de factorización de gramáticas: en vez de que dos producciones alternativas empiecen ambas por los mismos símbolos y dejen que el parser “adivine” cuál es, se las fusiona en una sola producción que consume el prefijo común y recién más adelante, con la gramática ya en un estado neutro, se bifurca según el símbolo siguiente. Es el mismo principio que la factorización por la izquierda en gramáticas LL, aplicado acá para evitarle al parser LALR(1) una decisión que no puede tomar con un solo token de adelanto.

6.2. Listas de hijos mediante NodeList

Para los nodos con una cantidad variable de hijos (por ejemplo, **VariableDeclaration**, cuya cantidad de identificadores declarados no se conoce de antemano), la interfaz de **ast.h** separa dos responsabilidades: **NodeList** es una lista enlazada transitoria, que solo existe durante la

construcción del árbol dentro de `bison.y` para ir acumulando nodos hermanos a medida que la gramática reduce reglas recursivas (`IdentifierList`, `VariableDeclarations`, `Statements`); y `attachChildren` es la función encargada de volcar esa lista al arreglo `children[]` del nodo padre una única vez, dejando el AST final compuesto solo por `NodeAST`, sin rastros de la lista auxiliar. Elegimos este esquema en dos pasos (acumular en lista, después volcar a arreglo) en lugar de, por ejemplo, usar directamente un arreglo dinámico que se va agrandando durante el parsing, porque encaja mejor con la forma en que Bison arma los valores semánticos de abajo hacia arriba en reglas recursivas por la derecha.

7. Testing

Agregamos tests unitarios para el lexer y para el parser, en la carpeta `tests/`, usando el framework Unity. Se corren con

```
bash test_suite.sh
```

que informa una línea `[PASS]` o `[FAIL]` por test. Cada test tiene un comentario que indica qué verifica.

En el lexer buscamos detectar tokens mal reconocidos: palabras reservadas confundidas con identificadores (mayúsculas o una palabra reservada pegada a otras letras), literales numéricos mal formados, operadores que no existen en C-TDS, comentarios que se cortan o no se ignoran, símbolos no permitidos y números de línea incorrectos.

En el parser verificamos que se acepten los programas válidos según la gramática y que se rechacen los que tienen errores de sintaxis: falta de punto y coma, cuerpos sin llaves, declaraciones y firmas mal formadas, sentencias y expresiones inválidas, llaves o paréntesis desbalanceados. También cubren la intercalación de variables y métodos globales (Sección 4), para detectar si ese cambio se rompe.

Los tests solo verifican reglas léxicas y sintácticas: un programa que viola únicamente una regla semántica se espera aceptado. Tampoco verifican la precedencia de operadores, porque sin el AST el parser solo informa si acepta o rechaza la entrada.

8. Problemas conocidos

8.1. Comentario multilínea sin cerrar

Detección: al construir casos de test para el lexer, probamos deliberadamente un archivo fuente donde un comentario multilínea se abre con `/*` y el archivo termina sin el `*/` correspondiente. La expresión regular actual de `lexer.1` para comentarios multilínea exige que el cierre exista, así que en este caso no matchea ninguna regla de comentario.

Caso de test:

```
/* este comentario  
nunca se cierra  
int nuncaSeEjecuta;
```

Comportamiento actual: como la regla de comentario no matchea (no hay `*/`), el lexer cae en las reglas siguientes y termina interpretando el contenido del comentario carácter por carácter, reportando cada símbolo dentro de él como error léxico, en lugar de señalar el problema real, que es que el comentario nunca se cerró.

Acción tomada: identificamos el problema y la solución conceptual (introducir un estado exclusivo de Flex, por ejemplo `%x COMMENT`, que se entre al ver `/*` y se salga al ver `*/`, junto con una regla de `«EOF»` dentro de ese estado que reporte explícitamente “comentario no cerrado”), pero decidimos no aplicarla todavía en `lexer.1`. Queda pendiente para una iteración posterior del lexer.

8.2. Alcance de esta entrega

Del AST, en esta etapa solo dejamos definida la interfaz (`ast.h`) y las invocaciones ya escritas pero comentadas dentro de `bison.y`, a modo de referencia de cómo se va a construir cada nodo; los cuerpos de `ast.c` están vacíos. No es un problema en el sentido de algo que falle: es una decisión deliberada (ver Sección 5), porque implementar el AST implica decisiones de diseño que todavía tenemos que cerrar como equipo. La próxima etapa consiste en completar los cuerpos de `ast.c` y descomentar las acciones semánticas de `bison.y` una vez que terminemos de acordar esos últimos detalles.